

# A Survey of Inconsistency in Heterogeneous LLM-Based Multi-Agent Systems

Xinyang Bao  
3035952989

Yu Wu  
3036509076

Jiaqi Zhong  
3035828528

**Abstract**—Large language models have transformed AI systems from passive text generators into autonomous agents capable of reasoning, planning, tool use, memory management, and environmental interaction. Building on these capabilities, LLM-based multi-agent systems have emerged as a promising paradigm for solving complex tasks through task decomposition, role specialization, and collaborative decision-making. However, real-world LLM-based MAS are often heterogeneous, involving agents with different model backbones, role prompts, memory mechanisms, tool permissions, communication styles, and task capabilities. While such heterogeneity improves flexibility and specialization, it also introduces inconsistency among agents. This survey examines inconsistency in heterogeneous LLM-based MAS through a Cognition–Memory–Action framework. Cognitive inconsistency refers to incompatible representations of tasks, states, goals, and intentions; memory inconsistency arises from divergent or outdated records of interaction histories, task progress, or shared facts; and action inconsistency occurs when agents produce conflicting communicative behaviors, tool calls, or environment-modifying actions. Based on this taxonomy, we analyze the sources, impacts, and mitigation mechanisms of inconsistency, and discuss open challenges including cross-model semantic grounding, scalable shared-state maintenance, heterogeneity-aware coordination, and safe tool execution.

**Index Terms**—inconsistency, heterogeneous LLM-based MAS

## I. INTRODUCTION

LLM-based agents have evolved from text generators into autonomous systems capable of reasoning, planning, tool use, memory management, and environmental interaction. However, in practical applications, LLM-based MAS are rarely homogeneous. Agents may differ in model backbones, role prompts, memory mechanisms, tool permissions, communication styles, and domain capabilities. Such heterogeneity improves flexibility and specialization, but it also introduces a critical reliability challenge: inconsistency. Different agents may interpret the same goal differently, maintain incompatible task histories, exchange incomplete information, or execute conflicting actions. As a result, locally reasonable agent behaviors may become globally incompatible at the system level.

This survey focuses on inconsistency in heterogeneous LLM-based MAS. We argue that inconsistency should be understood as a system-level phenomenon rather than an isolated failure of a single agent. Existing studies have examined LLM agents, multi-agent coordination, memory mechanisms, and tool use from different perspectives, but there remains limited systematic analysis of how heterogeneity produces inconsistency across the full MAS workflow. To address this gap, we propose a Cognition–Memory–Action framework that

organizes inconsistency according to three key stages of multi-agent collaboration: how agents understand tasks (cognition), how they maintain and update information (memory), and how they communicate or execute actions (action).

The rest of this survey is organized as follows. Section 2 introduces the background of LLM-based agents and heterogeneous MAS, and presents the Cognition–Memory–Action taxonomy in detail. Section 3 analyzes inconsistency across cognition, memory, and action, and reviews current alignment methods for mitigating these inconsistencies. Finally, Section 4 discusses open challenges and future directions, including trajectory-level evaluation, heterogeneity-aware coordination, standardized communication protocols, and safe external execution mechanisms.

## II. BACKGROUND AND TAXONOMY

### A. Background: The Evolution of Agentic Systems

In recent years, large language models have gradually evolved from static text generation models into autonomous agents capable of reasoning, planning, tool use, memory retrieval, and environment interaction [1]–[3]. Early LLMs were primarily designed to generate text through next-token prediction based on the given context. However, with the development of reasoning-action frameworks such as ReAct and tool-augmented methods such as Toolformer, LLMs have increasingly been used not only to produce linguistic outputs, but also to make decisions, call external tools, observe feedback, and adjust subsequent actions [2], [3]. As a result, the research focus has shifted from generating correct textual responses to enabling LLM-based agents to perform reliable actions in dynamic task environments.

However, single-agent systems often face clear limitations when handling complex and long-horizon tasks. A single agent may need to understand task requirements, maintain context, retrieve information, generate plans, call tools, verify intermediate results, and recover from errors at the same time. This design is vulnerable to limited context windows, memory loss, insufficient domain expertise, and error accumulation across multiple steps. LLM-based multi-agent systems address these limitations by distributing complex tasks among multiple agents through task decomposition, role specialization, communication, and collaborative decision-making [4], [5]. Recent surveys describe LLM-MAS as systems involving agent profiles, perception, self-action, mutual interaction, and evolution, suggesting that multi-agent collaboration is not a

simple combination of independent LLMs, but a structured workflow across multiple stages of agent behavior [5].

As LLM-based MAS move toward practical applications, their design is also shifting from homogeneous redundancy to heterogeneous specialization. Early multi-agent settings often used multiple similar agents to improve robustness through voting, debate, or self-consistency. In contrast, modern LLM-MAS increasingly combine agents with different model backbones, role prompts, memory mechanisms, tool permissions, communication protocols, and domain capabilities. For example, MetaGPT assigns agents with different professional roles and encodes Standardized Operating Procedures into multi-agent collaboration, while AutoGen allows developers to construct customizable conversable agents and flexible interaction patterns [6], [7]. These frameworks show that heterogeneity can improve specialization and workflow efficiency.

Nevertheless, heterogeneity also introduces new consistency challenges. Because agents may differ in their model capabilities, role profiles, memory access, communication styles, and tool-use permissions, they may interpret the same task differently, maintain incompatible memory states, or execute conflicting actions. Therefore, consistency in heterogeneous LLM-based MAS should not mean forcing all agents to reason identically. Rather, it requires agents to maintain sufficiently compatible understandings of task goals, shared facts, collaborative intentions, and execution constraints. Based on this background, this survey categorizes inconsistency in heterogeneous LLM-based MAS into three dimensions: cognition, memory, and action.

### B. Taxonomy of LLM-Based MAS Inconsistency

The central challenge in LLM-based MAS is the alignment gap among heterogeneous agents. Differences in model backbone, role profile, memory capacity, communication protocol, and tool-use capability can cause a local misunderstanding to propagate across the system. A single misinterpreted instruction or inconsistent memory update may therefore compromise the final task outcome. Recent surveys and representative multi-agent frameworks further suggest that inconsistency can emerge throughout the LLM-MAS workflow, including agent profiling, inter-agent communication, role specialization, and task execution, rather than only at the final action stage [4], [6], [7]. This paper organizes consistency issues in LLM-based MAS into three dimensions.

1) *Cognitive Inconsistency*: Cognitive inconsistency describes the extent to which heterogeneous agents maintain incompatible representations of tasks, environmental states, goals, and collaborative intentions. It weakens coordination because agents may exchange syntactically valid messages while assigning different meanings to the same symbols. In this sense, cognitive inconsistency is not a marginal issue but a direct obstacle to coherent multi-agent collaboration.

2) *Memory Inconsistency*: Memory inconsistency arises when agents hold divergent or outdated representations of past events, interaction histories, task progress, and shared environmental states. This includes discrepancies in episodic

memory (e.g., different agents recalling different outcomes of the same conversation), factual memory (e.g., conflicting records of task parameters), and procedural memory (e.g., differing recollections of previously negotiated workflows or role assignments). It also encompasses memory drift over time, selective forgetting, and inconsistent summarization of shared experiences, all of which erode the common factual foundation required for coordinated collaboration.

3) *Action Inconsistency*: Action inconsistency refers to conflicts or misalignments that emerge when agents transform their cognition and memory into observable operations. In LLM-based MAS, actions include both internal communicative actions, such as asking for clarification, and external execution actions, such as tool calls. Action inconsistency occurs when agents' actions are locally reasonable but globally incompatible, causing failures in coordination, execution, or shared environmental state. These failures may originate from upstream cognitive or memory divergence, from breakdowns in inter-agent communication, or from inconsistent interactions with external tools and environments.

To provide a clearer overview of the proposed taxonomy, Table I summarizes the three dimensions of inconsistency under the Cognition–Memory–Action framework. The table compares cognition, memory, and action inconsistency in terms of their definitions, main sources, typical manifestations, and potential impacts. This taxonomy serves as the structural foundation for the subsequent critical analysis, where each dimension is examined in more detail.

## III. CRITICAL ANALYSIS

### A. Cognition

1) *Sources and Impacts of Cognitive Inconsistency*: In heterogeneous LLM-based multi-agent systems (Fig. 1), cognitive consistency means that agents maintain compatible representations of tasks, states, goals, and intentions during collaboration. It does not require identical reasoning processes; rather, it requires that agents interpret the task in sufficiently compatible ways to support coordinated collaboration. This section identifies three sources of cognitive inconsistency.

1.1) *Model-level semantic interpretation divergence*. In practical LLM-MAS, agents may rely on different foundation models, fine-tuned models, or domain-specialized models. Because these agents can differ in model backbone, role profile, training data, and communication setting, they may ground the same instruction differently. For example, in a customer-service MAS (Fig. 2a), the instruction “handle an urgent order” may be represented by a logistics-oriented agent as “complete fulfillment within 30 minutes,” while a general dialogue agent may treat it as “respond today with high priority.” Such incompatible representations can produce task deviation, duplicated planning assumptions, and inconsistent subgoals. This issue is closely related to profiling, communication, and coordination challenges discussed in recent LLM-MAS surveys [4].

1.2) *Persona-driven cognitive misalignment*. Even when agents share the same base model, different role prompts can lead them to construct different cognitive frames for

TABLE I: Taxonomy of inconsistency in heterogeneous LLM-based MAS under the Cognition–Memory–Action framework.

| Dimension               | Definition                                                                                                       | Main Sources                                                                               | Typical Manifestations                                                                                 | Potential Impacts                                                                             |
|-------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Cognition inconsistency | Agents maintain incompatible representations of tasks, goals, states, or intentions.                             | Model-level semantic divergence; persona-driven misalignment; intention compression.       | Different interpretations of the same instruction; inconsistent sub-goals; mismatched task priorities. | Task deviation; duplicated planning; weak coordination; misunderstanding across roles.        |
| Memory inconsistency    | Agents hold divergent, outdated, or incomplete records of shared facts, interaction histories, or task progress. | Context-window limits; asynchronous updates; private memories; inconsistent summarization. | Conflicting task histories; outdated state records; missing shared facts; asymmetric knowledge.        | Wrong decisions; repeated work; broken collaboration; unstable shared beliefs.                |
| Action inconsistency    | Agents produce conflicting communicative behaviors, tool calls, or environment-modifying actions.                | Upstream cognition or memory conflicts; poor communication; invalid or stateful tool use.  | Ignored messages; conflicting tool calls; inconsistent API parameters; unsafe side effects.            | Execution failure; corrupted environment state; cascading errors; reduced system reliability. |

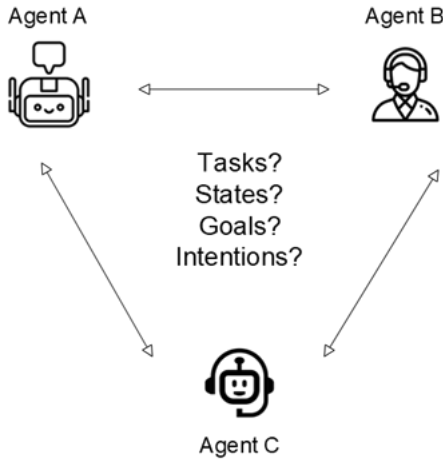


Fig. 1: Cognitive Consistency in LLM-MAS

the same task. In Fig. 2b, A logistics-oriented agent may interpret “urgent” primarily as a hard operational deadline, while a customer-success agent may interpret it as a need for reassurance and relationship repair. The problem is not role specialization itself; specialized roles are necessary in LLM-MAS. Rather, inconsistency arises when role prompts cause agents to prioritize different goals, constraints, or evaluation criteria without a shared task frame. In this sense, prompt heterogeneity becomes a cognitive issue because it reshapes how agents represent the task before any external action is taken. Recent studies show that prompt components influence diversity and consistency in LLM-agent conversations, while personality-driven agent research suggests that induced persona traits can affect planning and decision-making [8], [9].

1.3) Hierarchical intention compression. A third source of cognitive inconsistency arises when high-level intentions are transferred across roles, abstraction levels, or task interfaces. In LLM-MAS, a complex intention often needs to be con-

verted into a lower-level task specification before it can be used by another agent. During this conversion, the original intention may lose important semantic context. For example, a planner or coordinating agent may intend (Fig. 2c): “This VIP customer previously churned because of delivery delays; bypass the standard queue, use a dedicated courier, and notify the customer in real time.” However, this rich intention may be reduced to a sparse symbol such as `priority=true`. The downstream agent may preserve the surface instruction while losing the rationale, constraints, and expected outcome that made the instruction meaningful. This is a cognitive failure because the transmitted representation no longer preserves the original intention. Recent work on multi-agent communication also suggests that natural-language-only communication may lose latent reasoning information when internal reasoning states are compressed into discrete tokens [10].

2) *Existing Cognitive Alignment Mechanisms*: Existing alignment mechanisms rarely attempt to make heterogeneous agents think identically. Instead, they introduce external alignment structures that stabilize meaning across agents. These structures include shared world models, external state repositories, standardized workflows, task interfaces, and structured semantic representations. From the perspective of cognitive consistency, their common purpose is to externalize key semantic assumptions so that agents can coordinate shared task meanings, explicit intentions, and verifiable intermediate artifacts.

2.1) *Semantic Alignment through Shared World Models* The first strategy is semantic alignment through shared world models or shared external state. In this approach, agents do not rely only on private interpretation. Instead, they access a common representation of the task environment, including entities, constraints, resources, task status, and relevant execution context. In the urgent-order example (Fig. 3), the shared state may explicitly record that the order has VIP status, a 30-minute SLA, a warehouse-overload warning, and a real-time notification requirement. Once these facts are externalized, agents may

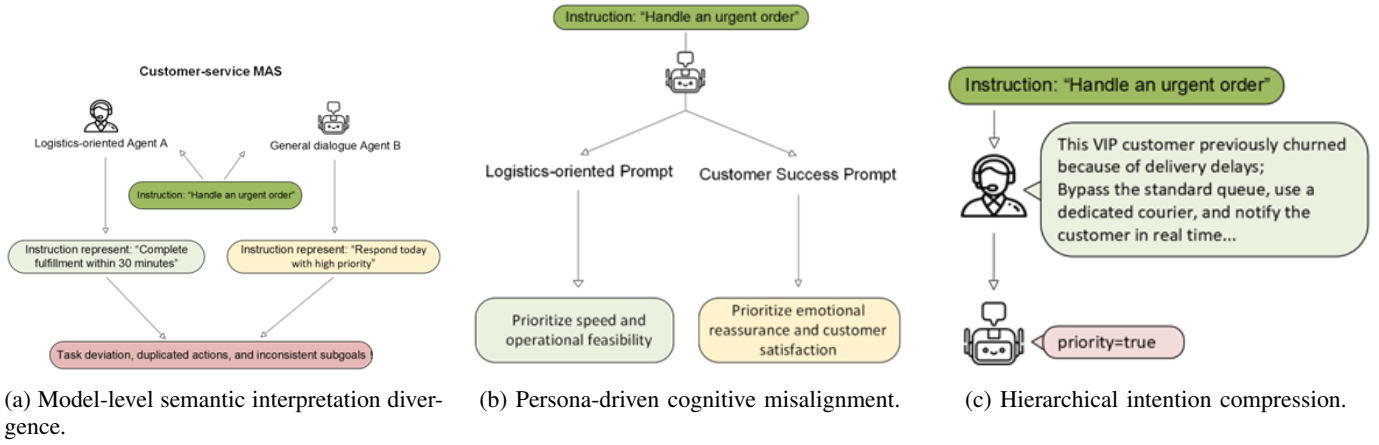


Fig. 2: Examples of three kinds of cognitive inconsistency in heterogeneous LLM-based MAS.

still reason differently, but they no longer reason from completely incompatible premises. Recent systems suggest that shared world models can serve as representation-consistency layers rather than passive state stores. DR. WELL provides a concrete example of how a dynamic symbolic world model may support this direction. It introduces a Dynamic Symbolic World Model represented as a Dynamic Symbolic Graph, where agents negotiate roles, generate symbolic plans, and update the shared model according to execution outcomes [11]. By recording assigned tasks, plan prototypes, plan instances, and execution logs, the DSG captures both the current task state and the history of collaboration. Here, execution traces are discussed as part of the shared representation for semantic alignment, not as low-level execution failures. By externalizing task state, planning structure, and execution traces, DSG-style world models can reduce semantic drift and support more stable inter-agent understanding.

This direction is also consistent with recent world-model alignment studies. WALL-E argues that LLMs’ prior knowledge may not naturally match the dynamics of a specific environment and proposes rule learning to align an LLM-based world model with agent-explored trajectories [12]. WALL-E 2.0 further extends this neurosymbolic direction by extracting symbolic knowledge, such as rules, knowledge graphs, and scene graphs, to improve the reliability of world-model-based agent reasoning [13]. Although these works focus more on agent-environment alignment than inter-agent consistency, they support the broader claim that symbolic and graph-based world models can provide stable semantic anchors for LLM agents.

2.2) Role and Intention Alignment through Standardized Workflows and Task Interfaces While shared world models align agents around common task facts, workflow-based mechanisms align how role-specific agents interpret, externalize, and hand off task intentions. This strategy is especially relevant to role-induced representational divergence: different prompts may legitimately make agents attend to different aspects of the same task, but standardized workflows constrain these

perspectives through shared role contracts, expected artifacts, and acceptance criteria.

MetaGPT is a representative example of this direction. It encodes Standardized Operating Procedures into prompt sequences and assigns agents specialized roles within an assembly-line workflow [6]. From a cognitive-consistency perspective, the key contribution of such SOPs is not merely task automation, but the externalization of role expectations: each agent knows what kind of representation it should produce, how that representation will be inspected, and how it should be interpreted by subsequent agents.

More recent work extends this SOP-oriented idea into flexible and programmable collaboration patterns. AutoGen introduces a multi-agent conversation framework in which agents are modeled as conversational entities (Fig. 4), and developers can specify interaction patterns through natural language instructions and code [7]. In cognitive terms, such programmable workflows serve as task interfaces: they make the boundaries between roles explicit and reduce the risk that one agent’s role-specific interpretation will silently drift away from the shared objective. For example, in a customer-service MAS with a standardized and protocolized workflow, the point is not the concrete execution of later actions, but the preservation of a shared task representation across role boundaries.

Therefore, workflow- and protocol-based mechanisms address prompt-induced inconsistency by bounding role specialization. Agents may still reason from different professional perspectives, but their outputs must pass through shared schemas, acceptance criteria, and verification points. This prevents role prompts from becoming isolated cognitive frames and helps ensure that intermediate artifacts still preserve the original user intention.

2.3) Structured Intermediate Cognition in Planner–Executor Architectures A final mechanism addresses hierarchical intention compression through structured intermediate representations. This mechanism is closely related to workflow-based alignment, but its focus is narrower: it specifies what

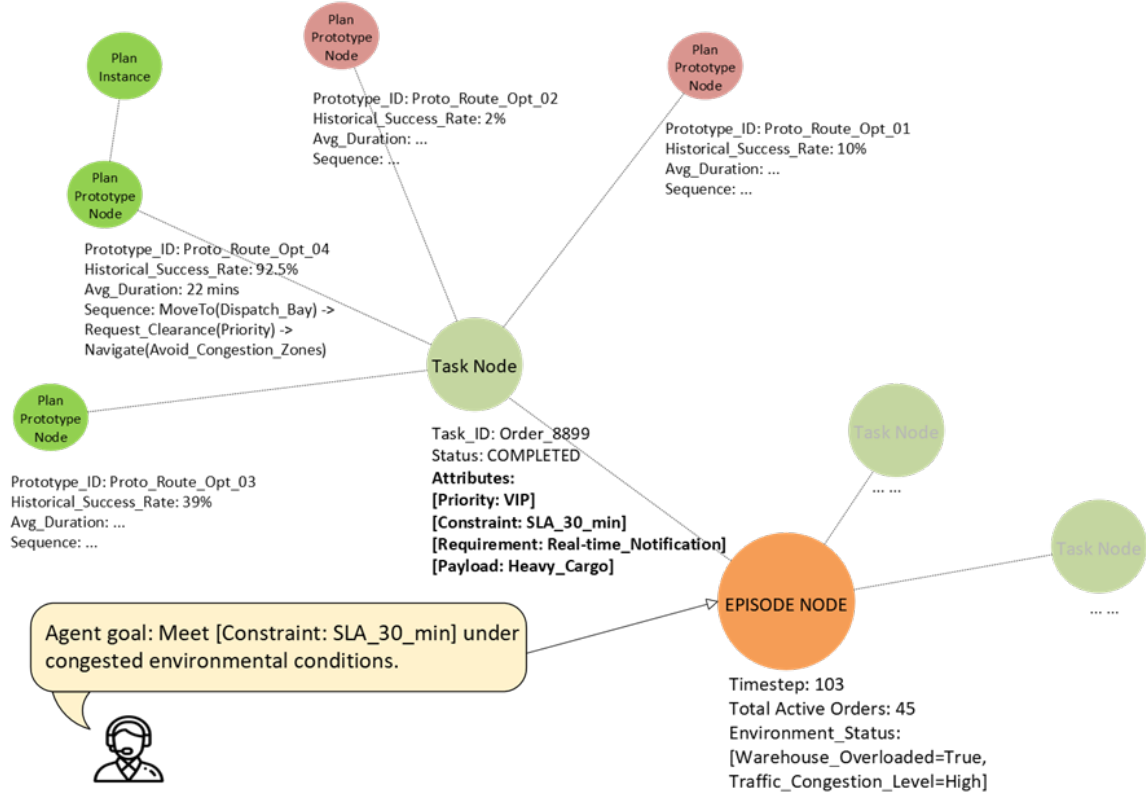


Fig. 3: Example of Customer-server MAS's Dynamic Symbolic Graph.

semantic information must be preserved when an intention is handed off across roles or abstraction levels. If a rich intention is compressed into a vague flag such as `priority=true`, downstream agents may preserve the surface instruction while losing the rationale that made it meaningful.

A structured task representation can instead require fields such as *goal*, *rationale*, *constraints*, *deadline*, *success criteria*, and *required notification*. For example, the urgent-order task may be represented as `SLA = 30 minutes`, `reason = VIP churn risk`, `bypass_queue = true`, and `notification = real-time`. In this sense, schema design functions as a semantic preservation mechanism rather than merely a syntactic validation tool. TalkHier and SEMAP support this direction by emphasizing structured communication, hierarchical refinement, behavioral contracts, and lifecycle-guided verification for preserving intention consistency across handoff and review stages [14], [15].

## B. Memory

1) *Key Limitation:* Contextual consistency serves as the cornerstone of reliable multi-round interactions for LLM-based systems, enabling agents to sustain factual accuracy, logical coherence, and decision-making stability throughout extended dialogues and long-horizon tasks. As emphasized in recent studies on multi-agent memory systems, contextual consistency is not merely a property of individual agent behavior but a foundational requirement for collective intelligence and

effective collaboration. However, native large language models are inherently constrained by their fixed context windows, which severely impede the maintenance of contextual consistency—especially in multi-agent settings—leading to three critical challenges.

First, information loss: key background information introduced in early dialogue stages is truncated or lost once interactions exceed the model's token capacity, eroding the shared context necessary for coordinated decision-making and long-term task execution. Second, content contradiction: agents may generate responses that conflict with historically valid information still retained within the context window, damaging reasoning reliability, and destabilizing collective beliefs. Third, information silos: in distributed multi-agent systems, the lack of structured memory synchronization and shared cognitive substrates causes agents to maintain inconsistent world states and asymmetric knowledge, directly breaking collaborative alignment and task coherence.

2) *Alignment Method:* In large language model (LLM) multi-agent systems, memory alignment serves as a core prerequisite for ensuring effective collaboration, avoiding task conflicts, and enhancing decision-making accuracy [16]. To mitigate memory inconsistency arising from independent memory maintenance, asynchronous interactions, and divergent information among agents, existing studies have established three mainstream memory alignment strategies. These approaches unify agents' knowledge perspectives, synchronize

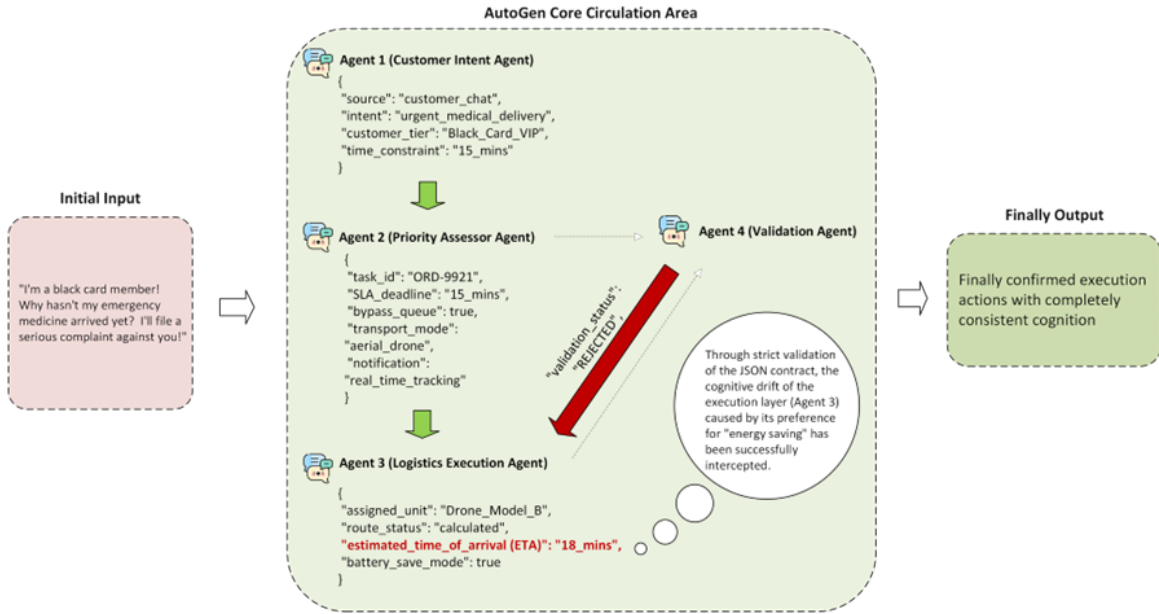


Fig. 4: Example of Customer-server MAS Based on AutoGen.

interaction histories, and maintain consistent global memory states from complementary viewpoints. Specifically, the three strategies include: shared memory as the unique source of truth, explicit synchronization protocols, and conflict resolution with consensus mechanisms. Each strategy features distinct design logic, implementation paradigms, and applicable scenarios, collectively offering comprehensive technical support for reliable memory alignment in LLM-based multi-agent systems. The following sections elaborate on each alignment strategy in detail.

2.1) Shared Memory as the Unique Source of Truth: To fundamentally eliminate information silos and view differences in multi-agent collaboration, the strategy of shared memory as the unique source of truth establishes a unified global memory base, which acts as the only authoritative data reference for all agents. This mechanism enforces agents to read from and write to the same memory layer, rather than maintaining independent local memory copies that may lead to inconsistency. The core design idea is to treat shared memory (including blackboard architecture, global knowledge bases, and unified shared storage) as the single source of truth for the entire system, ensuring all agents access consistent shared facts, interaction histories, and task states.

The Blackboard architecture is the most typical implementation of this strategy. As illustrated in Fig. 5, the system maintains a global public blackboard memory, where agents cannot privately maintain independent global memory copies. Instead, they only write observations and reasoning results to the blackboard, and read the current global task state, shared facts, and interaction history from it [17]. This architecture naturally unifies the knowledge views of all agents, making it widely used in multi-agent systems for task collaboration and distributed decision-making, and effectively avoids memory

inconsistency caused by independent local memory.

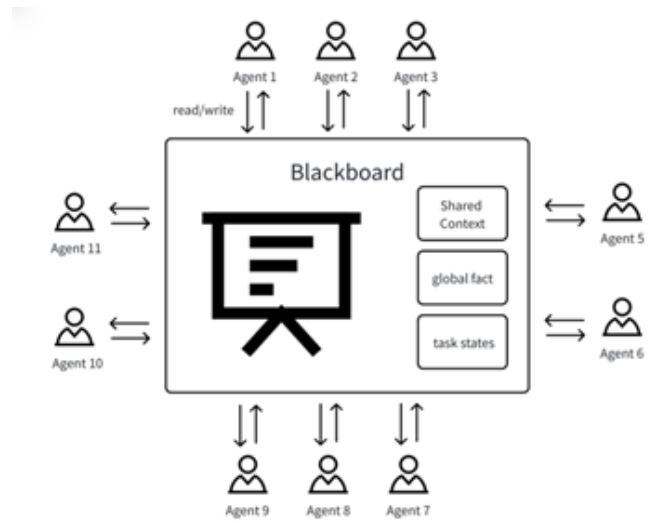


Fig. 5: Blackboard Architecture for Shared-Memory Based Memory Alignment

2.2) Explicit Synchronization Protocols: Unlike the shared-memory paradigm, explicit synchronization protocols achieve memory alignment without relying on a centralized global memory space, addressing inconsistencies caused by asynchronous updates and unordered interactions. Guided by the philosophy of "rule-based synchronization and incremental alignment", this mechanism standardizes the timing and methods of memory interaction through three core modules: synchronization trigger mechanisms (including periodic and event triggering), data transmission specifications, and conflict prevention rules. This enables dynamic alignment between

agents' local memory and the global system state.

The Publish/Subscribe (Pub/Sub) architecture is the most representative implementation of this approach. As shown in Fig. 6, a global memory message bus is established to decouple agents into publishers and subscribers. Publishers encapsulate memory updates into event messages and release them to the bus, while subscribers receive incremental data in real time to refresh their local memory. Atomic write rules and version number management further prevent dirty data and synchronization conflicts. In addition, periodic state update protocols and master-slave synchronization protocols serve as effective variants, adapting to different scenarios with varying consistency requirements and system scales.

Compared with strong-constraint shared memory alignment approaches, explicit synchronization protocols balance agent autonomy and memory consistency, reducing global storage and communication overhead while maintaining high alignment accuracy. Relevant studies [16] have verified that Pub/Sub-based explicit synchronization can reduce inter-agent memory inconsistency by over 40% and cut global communication overhead by 30%. Nevertheless, this alignment mechanism still has limitations, such as potential message bus congestion and synchronization delays in large-scale systems with high update frequencies, which can be mitigated through future optimizations like dynamic synchronization thresholds and semantic-level incremental synchronization to enhance its adaptability in complex LLM multi-agent memory alignment scenarios.

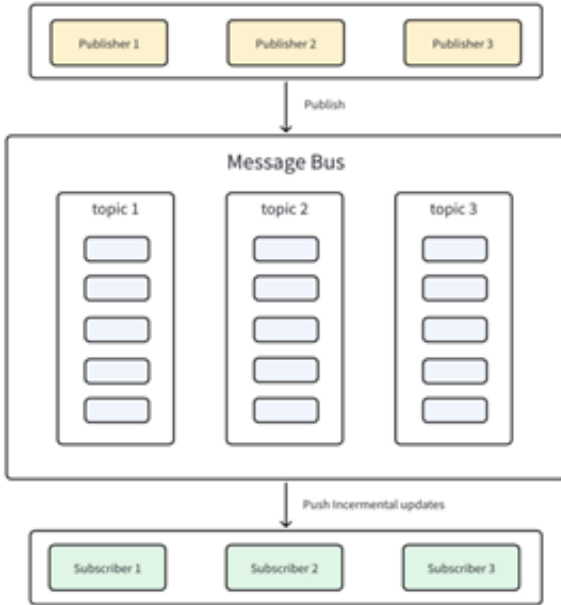


Fig. 6: Pub/Sub-Based Explicit Synchronization Architecture

2.3) Conflict Resolution and Consensus Strategies: When agents maintain independent local memory copies, inconsistencies inevitably arise due to divergent perceptions and up-

dates; conflict resolution and consensus strategies are designed to address this challenge while preserving agent autonomy. The core design concept is to allow agents to independently perceive and generate local memory, and when contradictory information is detected, a unified global consensus is reached through predefined arbitration rules, which is then broadcast to all agents to correct conflicting local memory.

Typical arbitration rules include timestamp priority, confidence weighting, role authority differentiation, and voting mechanisms, among which the distributed consensus architecture based on timestamp and confidence is a widely used implementation. In this architecture, each agent can independently generate local memory; when conflicting facts are detected, the system makes an arbitration based on “latest timestamp priority + weighted confidence of facts”; if the confidence is equal, the role authority weight of the agent (e.g., the weight of a supervisor agent is higher than that of an ordinary executive agent) is introduced to determine the only legitimate global fact, thereby maintaining the global memory consistency of the system.

Compared with shared memory and explicit synchronization protocols, this strategy offers greater flexibility in handling complex multi-agent interaction scenarios, especially in systems where agents have strong autonomy and frequent information updates. Relevant studies have shown that this consensus-based conflict resolution mechanism can effectively resolve more than 85% of memory conflicts in multi-agent systems [18]. However, this mechanism also has certain limitations, such as high computational overhead caused by consensus arbitration and potential delays in large-scale multi-agent systems, which can be optimized through lightweight consensus algorithms and semantic conflict detection technologies in future research.

### C. Action

This section systematically examines the inconsistency problems that may emerge at the action stage of multi-agent collaboration. Specifically, action inconsistency is further divided into three categories (Fig. 7): propagated action inconsistency, which originates from upstream cognitive and memory inconsistencies; internal action inconsistency, which arises from failures in inter-agent communication and coordination; and external action inconsistency, which occurs when agents interact inconsistently with tools, APIs, databases, or other external environments.

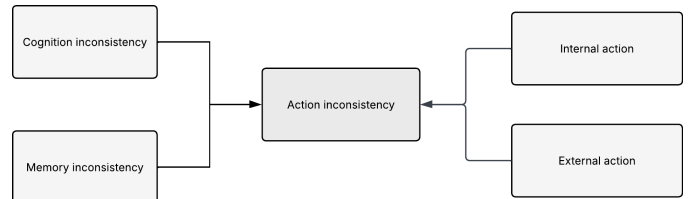


Fig. 7: Sources of Action Inconsistency

1) *Propagated Action Inconsistency from Cognitive and Memory Failures*: Propagated action inconsistency refers to action-level conflicts that originate from upstream cognition or memory inconsistency. In this case, the inconsistency does not first appear at the execution layer. Instead, divergent reasoning paths, conflicting beliefs, inconsistent task interpretations, or outdated memories are transformed into inconsistent plans, tool selections, or execution decisions. Formally, the action of an agent can be regarded as a function of its cognition and memory:

$$\begin{aligned} \text{Action}_A &\propto (\text{Cognition}_A \cup \text{Memory}_A) \\ &\neq \text{Action}_B \propto (\text{Cognition}_B \cup \text{Memory}_B). \end{aligned}$$

This formulation indicates that when Agent A and Agent B rely on different cognitive states or memory states, their action tendencies may also diverge. In other words, cognition and memory serve as upstream determinants of action generation; once inconsistency emerges at these upstream layers, it can be propagated into the execution layer and appear as inconsistent plans, tool selections, or operational decisions. For example, agents may adopt different understandings of the same goal, retrieve incompatible task states, or reason from conflicting assumptions, leading them to execute mutually incompatible actions. Therefore, propagated action inconsistency is often the downstream manifestation of deeper cognitive or memory-level inconsistency.

2) *Inconsistency by Internal Action*: Internal action inconsistency refers to failures in the communicative and coordinative actions among agents. In LLM-based MAS, agents perform not only external actions such as tool calls, but also internal actions such as sharing information, delegating tasks, asking for clarification, and responding to other agents' messages. Internal action inconsistency occurs when these interaction-level actions fail, causing agents to act on incomplete, ignored, or misaligned information.

Recent empirical evidence directly supports this type of internal action inconsistency. Cemri et al. identify *inter-agent misalignment* as a major failure category in LLM-based MAS, accounting for approximately 36.94% of observed failures [19]. This category includes conversation reset, failure to ask for clarification, task derailment, information withholding, ignored input from other agents, and reasoning-action mismatch. These failure modes suggest that MAS failures often arise not from the incapability of an individual agent, but from breakdowns in information transfer, clarification, coordination, and action alignment among agents.

For example, if one agent knows that a login API requires a phone number as the username but does not communicate this constraint to the supervisor agent, the supervisor may repeatedly attempt login with an email address and fail. This reflects information withholding; if the supervisor also fails to ask for clarification after repeated errors, the failure further involves insufficient repair of inter-agent communication. Such cases show that internal action consistency requires more than message exchange: agents must share task-critical information, incorporate other agents' inputs, ask clarification questions

when necessary, and ensure that their executed actions remain aligned with both their reasoning and the shared task state.

3) *Inconsistency by External Action (Tool-using)*: External action, especially tool use, can lead to action failure because it transforms an agent's output from a reversible linguistic act into an environment-modifying operation. Unlike internal reasoning or inter-agent communication, tool-using actions must satisfy external interface constraints, interact with stateful environments, and may produce persistent side effects. First, tool calls are vulnerable to parameter-level failures: an agent may select the correct tool but generate inaccurate arguments, unsupported API usage, or hallucinated parameters. Gorilla shows that even strong LLMs struggle with accurate API argument generation and may hallucinate API calls, indicating that valid tool invocation is a distinct capability rather than a guaranteed result of task understanding [20]. Second, real-world tool use is often stateful rather than stateless. ToolSandbox emphasizes that tool-use agents must handle stateful execution, implicit state dependencies between tools, and dynamic intermediate milestones, which means that an incorrect tool call can affect later actions and make subsequent reasoning operate on an inconsistent environment state [21]. Third, tool actions may have durable side effects. Atomix argues that LLM agents increasingly act on external systems where tool effects are immediate; under failures, speculation, or contention, unintended side effects may leak without safe rollback mechanisms [22]. Therefore, external action inconsistency arises because tool-use couples agent decisions with constrained, stateful, and side-effect-preserving environments: two agents may pursue semantically similar intentions, but differences in parameter validity, state awareness, or recovery behavior can cause one action to succeed while another fails or corrupts the shared environment.

4) *Current Alignment Methods for Action Inconsistency*: Besides maintaining consistency at the upstream cognition and memory stages, action consistency can also be improved at the action layer itself. Current alignment methods can be understood from two complementary perspectives: inter-agent alignment, which regulates how agents communicate, delegate, clarify, and respond to one another; and external-tool alignment, which improves the reliability, recoverability, and state safety of tool-using actions.

4.1) *Inter-Agent Alignment through Conversation Orchestration*: The first method focuses on aligning communicative actions among agents. AutoGen provides a relevant example by introducing a multi-agent conversation framework in which agents are customizable, conversable, and can operate with different combinations of LLMs, human inputs, and tools [7]. More importantly, AutoGen allows developers to define flexible agent interaction behaviors and conversation patterns through natural language or code. This turns free-form agent interaction into a programmable conversation process.

For action inconsistency, this is important because many internal failures arise when agents ignore each other's inputs, fail to ask for clarification, withhold task-critical information, or act on misaligned dialogue states. By specifying

roles, turn-taking patterns, tool-use permissions, and human-in-the-loop checkpoints, conversation orchestration can reduce coordination-related action inconsistency in heterogeneous MAS.

4.2) External-Tool Alignment through Failure Recovery and Transactional Control: The second kind of work addresses action inconsistency at the external execution level. Tool-using agents interact with APIs, databases, files, or other stateful environments, so their actions may produce persistent side effects. PALADIN focuses on one part of this problem: robust recovery from tool-use failures. It observes that tool-augmented language agents frequently fail in real-world deployment due to timeouts, API exceptions, or inconsistent tool outputs, which can trigger cascading reasoning errors and task abandonment [23]. PALADIN trains agents on failure-injected and recovery-annotated trajectories, making error detection and recovery action selection part of the agent’s learned capability rather than relying on ad hoc retry behavior.

Atomix addresses a complementary problem: unsafe side effects from external tool calls [22]. It argues that LLM agents increasingly act on external systems where tool effects are immediate, and that under failures, speculation, or contention, losing branches may leak unintended side effects without safe rollback. To mitigate this, Atomix introduces transactional semantics for agent tool calls, using mechanisms such as epoch tagging, resource-frontier tracking, delayed commits, and compensation on abort. This is especially relevant to external action inconsistency, because it helps prevent partially executed or conflicting tool actions from corrupting the shared environment state.

#### D. Comparative Summary of Alignment Methods

Table II summarizes representative alignment mechanisms for mitigating inconsistency in heterogeneous LLM-based multi-agent systems. Rather than forcing agents to behave identically, these methods aim to maintain compatible task understanding, synchronized memory states, and coordinated actions across heterogeneous agents. The comparison highlights that each mechanism addresses a specific layer of the Cognition–Memory–Action framework and involves distinct trade-offs among consistency, scalability, autonomy, and adaptability.

Overall, cognition-level methods mainly externalize task meaning, memory-level methods maintain shared or synchronized factual states, and action-level methods regulate communication and tool execution. This comparison suggests that inconsistency mitigation in heterogeneous LLM-based MAS requires layered alignment rather than a single universal solution.

### IV. OPEN CHALLENGES AND FUTURE DIRECTIONS

#### A. Cognition

Despite existing alignment mechanisms, cognitive consistency in heterogeneous LLM-MAS remains unresolved. First, cross-model semantic grounding is still a fundamental challenge. Shared states and structured protocols can reduce

ambiguity, but they do not create a truly unified semantic space across different LLMs. Agents may still assign different operational meanings to abstract terms such as “urgent,” “complete,” or “safe.” Future work therefore needs better methods to evaluate whether agents ground the same instruction in compatible task constraints, rather than merely comparing their final textual outputs.

Second, many alignment mechanisms lack dynamic adaptation. Prompts, schemas, role descriptions, and SOPs are often fixed before deployment, while real MAS environments involve changing user preferences, resource constraints, and unexpected edge cases. Recent surveys identify adaptation, communication, and coordination as key challenges for LLM-MAS [4].

Third, scalability remains difficult. As the number of agents increases, shared world models and structured protocols become harder to synchronize, query, and maintain. Finally, emergent inconsistency may arise through interaction, even when individual agents are well-aligned. Agents may reinforce incorrect assumptions or converge on misleading group interpretations. Therefore, future evaluation should treat cognitive consistency as a system-level property across multi-round collaboration, which is consistent with recent benchmarks that evaluate collaboration and competition dynamics rather than isolated single-agent performance [24].

#### B. Memory

Despite the effectiveness of existing memory alignment strategies, several critical challenges remain for large-scale LLM multi-agent systems. High-concurrency scalability remains a bottleneck, as frequent memory updates lead to contention in shared memory structures, congestion in message buses, and excessive overhead in consensus mechanisms [16]. Meanwhile, most approaches only ensure syntactic consistency while failing to resolve semantic divergence in unstructured reasoning traces and contextual beliefs generated by LLMs [18]. Furthermore, strict privacy constraints in sensitive applications conflict with transparent information sharing in shared memory and explicit synchronization frameworks [17]. Finally, heterogeneous agents with diverse capabilities and authority levels demand adaptive alignment policies rather than static rules, limiting the flexibility and robustness of current designs.

#### C. Action

The first challenge is the lack of evaluation methods that directly measure action consistency across the whole execution process. Existing evaluations often emphasize final task success, while overlooking whether intermediate actions remain consistent with agent roles, prior communication, shared task states, and tool constraints. Recent failure analyses further motivate this need: Cemri et al. show that many MAS failures arise from inter-agent misalignment, including conversation reset, information withholding, ignored inputs, and reasoning–action mismatch [19]. Future benchmarks should therefore assess trajectory-level properties such as role compliance,

TABLE II: Comparison of alignment and mitigation methods for inconsistency in heterogeneous LLM-based MAS.

| Method                                                    | Targeted Inconsistency        | Main Idea                                              | Strengths                                               | Limitations                                      | Suitable Scenarios                              |
|-----------------------------------------------------------|-------------------------------|--------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------|-------------------------------------------------|
| Shared world models / external state repositories         | Cognitive inconsistency       | Maintain a common task and environment state.          | Reduces semantic drift; makes assumptions explicit.     | Hard to scale; may become outdated.              | Planning, customer service, robotics.           |
| Standardized workflows / SOPs                             | Cognitive inconsistency       | Use predefined procedures and handoff rules.           | Clarifies roles; improves collaboration predictability. | Rigid for open-ended tasks; needs manual design. | Software development, document generation.      |
| Structured task interfaces / intermediate representations | Cognitive inconsistency       | Represent intentions with structured fields.           | Preserves goals, constraints, and rationale.            | Schemas may miss tacit reasoning.                | Planner-executor and delegation tasks.          |
| Shared memory / blackboard architecture                   | Memory inconsistency          | Use one shared memory as the source of truth.          | Reduces information silos; unifies task history.        | Central bottleneck; privacy concerns.            | Small or medium MAS needing strong consistency. |
| Explicit synchronization protocols / Pub/Sub              | Memory inconsistency          | Synchronize local memories through update events.      | Balances autonomy and consistency.                      | Possible delay, missed updates, or congestion.   | Distributed MAS with frequent updates.          |
| Conflict resolution / consensus mechanisms                | Memory inconsistency          | Resolve conflicts by rules or voting.                  | Handles contradictions while preserving autonomy.       | Extra overhead; may reinforce wrong facts.       | Autonomous agents with uncertain observations.  |
| Conversation orchestration                                | Internal action inconsistency | Regulate turns, roles, permissions, and clarification. | Reduces ignored inputs and coordination failures.       | May limit flexibility; needs careful design.     | Multi-agent chat, coding, customer service.     |
| Tool-use validation and schema checking                   | External action inconsistency | Validate tool calls before execution.                  | Prevents invalid calls and hallucinated parameters.     | Cannot fix wrong reasoning or tool choice.       | API, database, and workflow agents.             |
| Failure recovery mechanisms                               | External action inconsistency | Detect failures and trigger recovery actions.          | Reduces cascading errors in long tasks.                 | May cause costly retries or loops.               | Web automation, coding, data analysis.          |
| Transactional control / rollback mechanisms               | External action inconsistency | Use logs, delayed commits, rollback, or approval.      | Prevents unsafe side effects.                           | Complex; not all tools support rollback.         | Databases, payments, file systems, APIs.        |

clarification behavior, information-sharing completeness, plan-action consistency, tool-call validity, and conflict recovery, rather than only judging the final answer.

The second challenge is that current multi-agent systems often rely on static roles or fixed conversation protocols. Although frameworks such as AutoGen make agent interaction programmable [7], heterogeneous agents may still differ in model capability, latency, memory access, tool availability, authority level, and reliability. These differences can cause agents to ignore others’ inputs, withhold task-critical information, or act on outdated dialogue states. Future systems should develop adaptive coordinators that assign tasks according to agent capability, monitor whether key information has been shared, trigger clarification when needed, and prevent conflicting or duplicated actions. This would extend conversation orchestration from static role design to dynamic coordination.

A third challenge concerns external action inconsistency caused by tool use. Tool-using actions interact with constrained, stateful, and side-effect-preserving environments, so failures may corrupt the shared state rather than merely produce an incorrect answer. ToolSandbox shows that realistic tool use involves stateful execution and implicit dependencies between tools [21], while Atomix argues that immediate tool effects may leak unsafe side effects under failures, speculation, or contention [22]. Future systems should treat tool actions as controlled state transitions, using schema validation, precondition checks, transaction logs, postcondition verification, rollback, compensation, and human approval for high-risk actions. This can make external execution more recoverable and consistent.

## V. CONCLUSION

This survey examined inconsistency in heterogeneous LLM-based multi-agent systems through a Cognition–Memory–Action framework. Instead of treating inconsistency as an isolated failure of individual agents, the proposed framework organizes it as a system-level problem that emerges across task understanding, information maintenance, and action execution.

The analysis shows that cognitive, memory, and action inconsistency are closely interrelated. Divergent task interpretations may lead agents to maintain incompatible memories, while outdated or conflicting memory states can further propagate into inconsistent communication, tool calls, or environment-modifying actions. Therefore, consistency in heterogeneous LLM-MAS should not require agents to reason identically, but should ensure that their task representations, shared facts, and execution constraints remain sufficiently compatible for coordinated collaboration.

Existing mitigation mechanisms provide useful partial solutions, including shared world models, standardized workflows, structured task interfaces, shared memory, synchronization protocols, consensus mechanisms, conversation orchestration, and transactional tool control. However, these methods still face limitations in cross-model semantic grounding, scalable shared-state maintenance, adaptive coordination, privacy-preserving memory synchronization, and safe external execution. Future research should therefore move beyond static and rule-based alignment toward dynamic, scalable, and system-level mechanisms that can maintain consistency throughout the full multi-agent collaboration process.

## ACKNOWLEDGMENT

We would like to express our sincere gratitude to our supervisor, Prof. Edith Cheuk-Han Ngai, for her invaluable guidance and continuous support throughout the research and manuscript preparation. Her expertise in artificial intelligence, multi-agent systems, and large language models offered critical insights for the framework design and systematic analysis of this survey, and her rigorous academic supervision facilitated the refinement of our research methodology and the enhancement of manuscript quality.

We also acknowledge Shuo Yang, Teaching Assistant, for his constructive feedback and technical assistance during the revision. His careful review and insightful comments on content organization and logical coherence contributed significantly to the improvement of this survey.

## REFERENCES

- [1] L. Wang *et al.*, “A survey on large language model based autonomous agents,” *arXiv preprint arXiv:2308.11432*, 2023.
- [2] S. Yao *et al.*, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [3] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [4] T. Guo *et al.*, “Large language model based multi-agents: A survey of progress and challenges,” *arXiv preprint arXiv:2402.01680*, 2024.
- [5] X. Li *et al.*, “A survey on llm-based multi-agent systems: Workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, p. 9, 2024.
- [6] S. Hong *et al.*, “Metagtpt: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations*, 2024.
- [7] Q. Wu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversations,” in *First Conference on Language Modeling*, 2024.
- [8] K. C. Chu, Y. P. Chen, and H. Nakayama, “Exploring and controlling diversity in llm-agent conversation,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [9] L. Newsham and D. Prince, “Personality-driven decision-making in llm-based autonomous agents,” in *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, 2025.
- [10] Y. Tang, W. Su, Y. Zhou, Y. Liu, M. Zhang, S. Ma, and Q. Ai, “Augmenting multi-agent communication with state delta trajectory,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- [11] N. Nourzad, H. Yang, S. Chen, and C. Joe-Wong, “Dr. well: Dynamic reasoning and learning with symbolic world model for embodied llm-based multi-agent collaboration,” *arXiv preprint arXiv:2511.04646*, 2025.
- [12] S. Zhou, T. Zhou, Y. Yang, G. Long, D. Ye, J. Jiang, and C. Zhang, “Wall-e: World alignment by rule learning improves world model-based llm agents,” *arXiv preprint arXiv:2410.07484*, 2024.
- [13] —, “Wall-e 2.0: World alignment by neurosymbolic learning improves world model-based llm agents,” *arXiv preprint arXiv:2504.15785*, 2025.
- [14] Z. Wang, S. Moriyama, W.-Y. Wang, B. Gangopadhyay, and S. Takamatsu, “Talk structurally, act hierarchically: A collaborative framework for llm multi-agent systems,” *arXiv preprint arXiv:2502.11098*, 2025.
- [15] Z. Mao, J. Keung, F. Zhang, S. Liu, Y. Wang, and J. Li, “Towards engineering multi-agent llms: A protocol-driven approach,” in *Proceedings of the 32nd Asia-Pacific Software Engineering Conference, Early Research Achievements Track*, 2025.
- [16] S. Wu and K. Shu, “Memory in llm-based multi-agent systems: Mechanisms, challenges, and collective intelligence,” *TechRxiv*, 2025.
- [17] A. Salemi, M. Parmar, P. Goyal, Y. Song, J. Yoon, H. Zamani, T. Pfister, and H. Palangi, “Llm-based multi-agent blackboard system for information discovery in data science,” *arXiv preprint arXiv:2510.01285v2*, 2026.
- [18] B. Duong, “Pbft-backed semantic voting for multi-agent memory pruning,” *arXiv preprint arXiv:2506.17338v2*, 2025.
- [19] M. Cemri *et al.*, “Why do multi-agent llm systems fail?” *arXiv preprint arXiv:2503.13657*, 2025.
- [20] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive apis,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 126 544–126 565.
- [21] J. Lu *et al.*, “Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 1160–1183.
- [22] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, “Atomix: Timely, transactional tool use for reliable agentic workflows,” *arXiv preprint arXiv:2602.14849*, 2026.
- [23] S. V. Vuddanti *et al.*, “Paladin: Self-correcting language model agents to cure tool-failure cases,” *arXiv preprint arXiv:2509.25238*, 2025.
- [24] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, Z. Wang, Z. Wang, C. Qian, X. Tang, H. Ji, and J. You, “Multiagentbench: Evaluating the collaboration and competition of llm agents,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.